

Agentic RAG

End-to-end walkthrough of the **canonical production pipeline**: clients → FastAPI → runner → canonical graph v1 *or* source-tools graph → response. Written from live code paths in `src/`.

What this document is for. Keep it beside the repo. Every major step names the function, file, and decision. If code and this manual diverge, trust the code — then update this file. Companion index: `docs/DOCUMENTATION_MAP.md`.

Edition: September 2026 (canonical v1)

Pipeline version: `canonical-v1`

Public modes: `canonical`, `source_tools`

Canonical entry: `run_agent()` → `ask_canonical()` or `ask_source_tools()`

Stack: LangGraph · LangChain · FastAPI · Chroma · OpenAI · Redis · React

Contents

1. How to read this document
2. System at a glance
3. Layered architecture
4. End-to-end request lifecycle
5. Conversation memory
6. Canonical LangGraph (production)
7. LLM agents vs deterministic functions
8. Ingestion & retrieval
9. Generation, verification, citations, streaming
10. Production envelope & Phase 8 evaluation
11. API surface & deployment
12. Configuration map
13. File ownership quick reference
14. Known gaps
15. Tests that document behavior
16. Worked traces

1. How to read this document

Start with §2–§4 for the whole picture. §6 is the canonical graph (replaces the retired eight-mode multi-graph runtime). §8–§9 cover retrieval and verification. §10–§11 for ops.

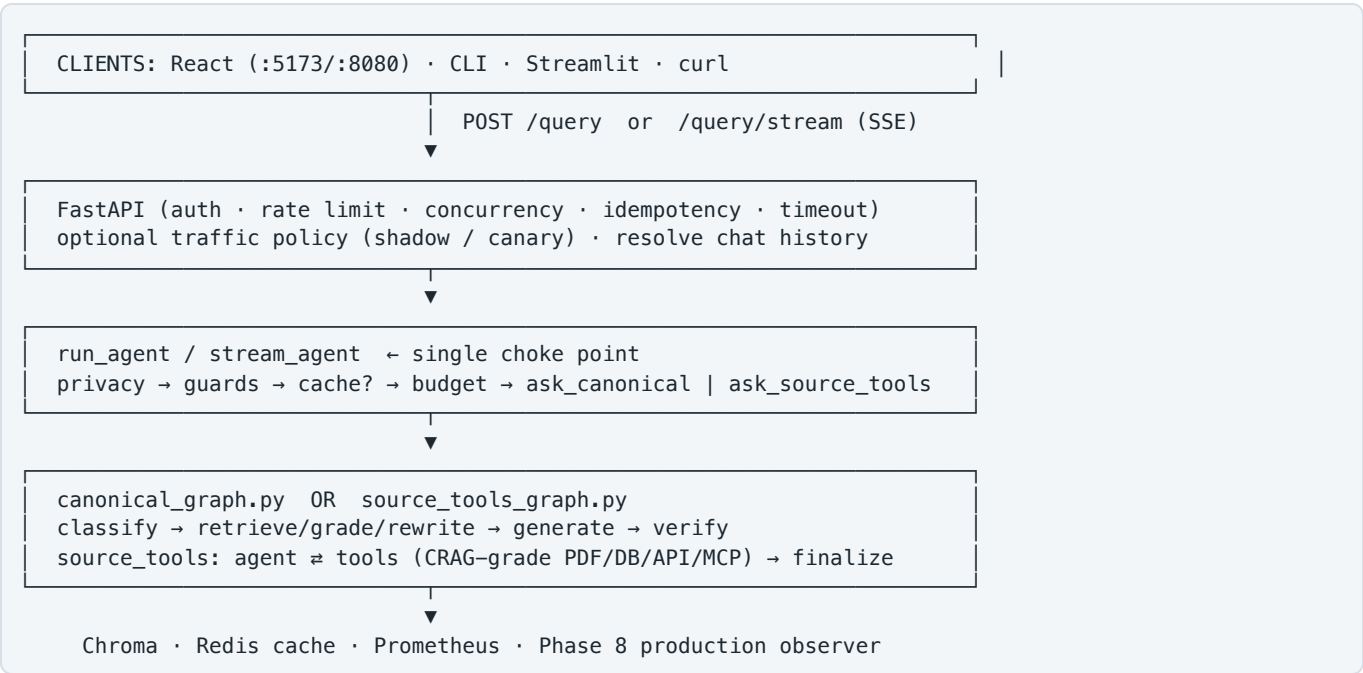
If you need...	Go to
One diagram of the whole product	§2 System at a glance
Exact steps on every API call	§4 Request lifecycle
Graph nodes, edges, state	§6 Canonical graph
What is an agent vs a function	§7 Agent boundaries
PDF → Chroma → hybrid retrieve	§8 Ingestion & retrieval
Verification & abstention	§9 Verification
Shadow/canary/continuous eval	§10 Phase 8

HISTORICAL NOTE

Phases 1–7 originally built eight separate mode graphs (`router_graph` , `crag_graph` , ...). Those modules were **removed** in Phase 7. Deprecated mode strings still accepted by the API map to canonical strategies internally. See `docs/ROADMAP.md` and `docs/LEGACY_RETIREMENT_INVENTORY.md` .

2. System at a glance

Agentic RAG is a **single canonical LangGraph workflow** with strategy selection, evidence grading, verification, and production controls (auth, cache, guardrails, continuous evaluation). One question flows through `classify` → `retrieve` (or `direct/web`) → `generate` → `verify` → `finalize`.



Mode key	Status	Behavior
canonical	Production	Full canonical graph; strategy selected inside graph.
source_tools	Production	LLM picks PDF / SQLite / ops API / lab MCP / calculator; source hits are CRAG-graded.
baseline, router, crag, ...	Deprecated	Accepted for compatibility; mapped to force_strategy; deprecation metrics emitted.

3. Layered architecture

Layer	Responsibility	Primary modules
1. Interface	HTTP, CLI, UI, SSE	src/api/server.py, documents.py, frontend/
2. Control plane	Privacy, guards, cache, memory, budgets	runner.py, guardrails.py, cache/
3. Graphs	Canonical workflow + optional source-tools ReAct	graph/canonical_graph.py, source_tools_graph.py
4. Knowledge plane	Ingest, hybrid retrieve, rerank, federation	ingestion/, retrieval/, sources/
5. Model & tools	LLM, chains, safe web fetch	llm.py, chains/, tools/safe_web.py
6. Evaluation	Shadow, canary, continuous eval, gates	evaluation/, /ops/

HARD RULE

Never call ask_canonical() or graph nodes from a new interface directly. Always go through run_agent() or stream_agent().

4. End-to-end request lifecycle

4.1 Master flow

```
Client
→ middleware (RequestId, CORS, TrustedHost?)
→ verify_api_key + enforce_client_rate_limit
→ (sync) Idempotency-Key lookup
→ _resolve_chat_history
→ concurrency semaphore
→ run_agent / stream_agent
  A. PrivacyGuard.apply_input
  B. InputGuardrails.validate (+ injection scan)
  C. should_use_cache?
  D. augment_question_with_history
  E. cache hit? → post-guards → return
  F. CostGuardrails + record_query
  G. _dispatch → ask_canonical()
  H. OutputGuardrails + privacy output scan
  I. optional QualityGuardrails
  J. generate_follow_ups
  K. set_cached_response (pipeline_version keyed)
  L. production_observer hook (Phase 8)
→ _persist_exchange (Supabase if configured)
→ QueryResponse / SSE
```

4.2 Inside `_dispatch`

File: `src/runner.py`. Dispatch:

```
normalized = normalize_mode(mode)
if normalized == "source_tools":
    return ask_source_tools(question)
strategy = resolve_canonical_strategy(normalized) # deprecated aliases → strategy
canonical = ask_canonical(question, force_strategy=strategy)
return canonical_to_api_response(canonical, display_mode="canonical")
```

On canonical failure with `legacy_runtime_enabled=false` (default): safe abstention response — no legacy graph replay.

4.3 Streaming — POST /query/stream

SSE <code>type</code>	Meaning
<code>step</code>	Graph progress (classify, retrieve, grade, ...)
<code>pipeline</code>	Structured debug payload for frontend pipeline panel
<code>token</code>	Answer token stream
<code>answer</code>	Full answer after generation
<code>sources</code>	Source labels + citations array
<code>follow_ups</code>	Grounded follow-up questions
<code>done</code>	Terminal: latency, session, verification fields, <code>pipeline_version</code>
<code>error</code>	Guardrail or runtime failure

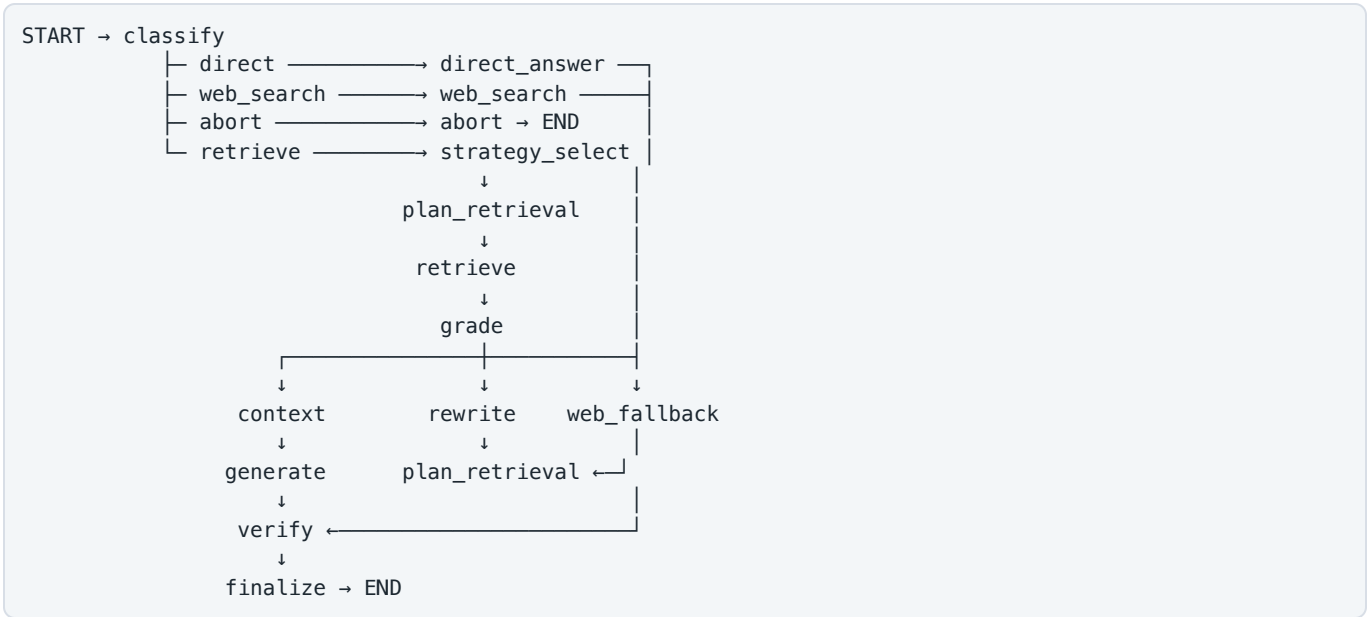
5. Conversation memory

Memory packing in `src/memory/chat_memory.py` ; optional Supabase persistence in `supabase_store.py` .

1. Recent N exchanges: full Q+A (truncated).
2. Older exchanges: questions only (topic trail).
3. Wrapped into `augment_question_with_history()` before `ask_canonical` .
4. Cache skipped when memory + non-empty history (answers must not leak across sessions).

6. Canonical LangGraph (production)

File: `src/graph/canonical_graph.py` · **State:** `CanonicalAgentState (src/contracts/state.py)` ·
Builder: `build_canonical_graph()`



Node	Kind	Module
<code>classify</code>	LLM router	<code>shared_nodes.invoke_router</code>
<code>strategy_select</code>	Heuristic + LLM	<code>strategy_heuristics.select_strategy</code>
<code>plan_retrieval / retrieve</code>	Deterministic	<code>retrieval/canonical_adapter.py</code>
<code>grade</code>	LLM grader	<code>evidence_management.py</code>
<code>rewrite</code>	LLM rewriter	<code>agents/query_rewriter.py</code>
<code>context</code>	Deterministic	<code>context_builder.py</code>
<code>generate</code>	LLM	<code>chains/generation.rag_chain</code>
<code>verify</code>	Rules + optional LLM	<code>verification_engine.py</code>
<code>finalize</code>	Deterministic	<code>citation_mapper.py</code> , API mapping

Strategies (internal, not separate graphs): `simple` , `decompose` , `multi_hop` . Deprecated API mode strings can force a strategy via `runner_modes.resolve_canonical_strategy` .

7. LLM agents vs deterministic functions

LLM decision components	Deterministic pipeline
Router (<code>classify_query</code>)	Hybrid dense + BM25 + RRF fusion
Strategy selector	Retrieval plan execution
Document grader	Cross-encoder rerank (NVIDIA / FlashRank)
Query rewriter	Parent-child expansion, deduplication
Generator	Token budgeting, citation index mapping
Verifier (optional LLM judge)	SSRF-safe web fetch, RBAC filters

TWO PRODUCTION GRAPHS
Canonical path: extra sources federate into `retrieve()` — no LLM tool picker. `mode=source_tools` is a separate ReAct graph: the LLM chooses PDF / DB / API / MCP / calculator; PDF/DB/API/MCP hits are CRAG-graded before they become citations. Calculator is not graded.

8. Ingestion & retrieval

8.1 Ingestion

```
python -m src.ingestion.ingest --source data/sample_docs
→ cleanse → section_parent_child chunking → embed → Chroma upsert
→ parent_store.json → invalidate BM25 + flush cache
```

Async jobs: `POST /ingest/jobs`, upload: `POST /ingest/upload`. Registry: `document_registry.py`.

8.2 Retrieval (deterministic)

```
query
→ over-fetch candidate_k (hybrid: dense + BM25 → RRF)
→ optional rerank (nvidia API or flashrank local)
→ RBAC tenant/role filters
→ optional federation (SQLite / ops API / lab MCP)
→ parent expansion → evidence candidates for grading
```

Entry: `execute_retrieval_plan()` in `src/retrieval/canonical_adapter.py`.

9. Generation, verification, citations, streaming

- **Generation:** LCEL chains in `src/chains/generation.py`; streaming via `stream_text`.
- **Verification:** Always runs before finalize. Outcomes: `PASS`, `WARN`, `FAIL`. May abstain or append caveat.
- **Citations:** `citation_mapper.map_citations` links evidence IDs to user-facing `Citation` objects.
- **Response status:** `answered` | `answered_with_warning` | `abstained`.
- **API bridge:** `canonical_to_api_response()` in `src/contracts/api_response.py`.
- **Follow-ups:** `generate_follow_ups` attached by runner after successful non-abort runs.

10. Production envelope & Phase 8 evaluation

Concern	Where
Auth + rate limits	<code>api/security.py</code> , <code>api/rate_limit.py</code>
Injection / PII	<code>security/injection.py</code> , <code>privacy.py</code>
Exact + semantic cache	<code>cache/redis_cache.py</code> , <code>semantic_cache_redis.py</code> (pipeline-version keyed)
Shadow / canary	<code>evaluation/shadow_*.py</code> , <code>canary_rollout.py</code>
Continuous eval	<code>evaluation/continuous_eval.py</code> , <code>eval_gates.py</code>
Quality dashboard	<code>GET /ops/quality/dashboard</code>
Legacy retirement status	<code>GET /ops/legacy/retirement</code>
Feedback loop	<code>POST /feedback</code> , <code>feedback_classifier.py</code>
LangSmith	Parent span <code>agent_request:<mode></code> nesting retrieve, grade, graph, follow-ups

Defaults: `CANONICAL_PRIMARY=true`, `LEGACY_RUNTIME_ENABLED=false`, `CONTINUOUS_EVAL_ENABLED=true`.

11. API surface & deployment

Method	Path	Purpose
GET	/health , /health/ready	Liveness / readiness
GET	/metrics	Prometheus
POST	/query , /query/stream	Sync / SSE query
GET	/modes , /config	Mode list + server config
GET/DELETE	/documents	Index management
POST	/ingest/upload , /ingest/jobs	Ingestion
POST	/feedback	User feedback
GET	/ops/preflight , /ops/canary/* , /ops/eval/*	Operator tools

```
docker compose up -d          # API :8000, frontend :8080, Redis, Chroma
uvicorn src.api.server:app --reload --port 8000 # local dev
```

12. Configuration map

All settings: `src/config.py` via `pydantic-settings`. Key groups:

Group	Examples
Canonical	CANONICAL_PRIMARY , LEGACY_RUNTIME_ENABLED , canonical_verification_llm_enabled
Source-tools	SOURCE_TOOLS_MAX_ROUNDS
Observability	LANGSMITH_TRACING , LANGSMITH_PROJECT
Retrieval	RETRIEVAL_SEARCH_TYPE , RETRIEVAL_TOP_K , RERANK_PROVIDER
Security	API_KEY , REQUIRE_API_KEY , TRUST_CLIENT_RBAC
Cache	CACHE_ENABLED , REDIS_URL , semantic cache thresholds
Eval / canary	SHADOW_ENABLED , CANARY_ENABLED , CONTINUOUS_EVAL_ENABLED
Memory	MEMORY_* , SUPABASE_*

13. File ownership quick reference

Path	Owns
src/runner.py	Choke point: guards, cache, dispatch to canonical
src/graph/canonical_graph.py	Default production LangGraph
src/graph/source_tools_graph.py	LLM-selected sources + CRAG grade
src/observability.py	LangSmith parent spans
src/contracts/	State, evidence, verification, API response mapping
src/retrieval/canonical_adapter.py	Deterministic retrieval boundary
src/evaluation/	Shadow, canary, continuous eval, regression gates
src/api/server.py	HTTP routes, SSE, ops endpoints
frontend/	Chat UI, pipeline debug panel, documents, feedback

14. Known gaps

Item	Reality
Idempotency	Sync /query only, not stream
Quality guardrails	Off by default (extra LLM cost)
CLI / Streamlit	Bypass HTTP auth, rate limits, concurrency
BM25 index	Per-process; not shared across API workers
Benchmark numbers	Not published in docs — run evaluators locally

15. Tests that document behavior

File	Documents
tests/test_source_tools_graph.py	Tool-selected sources + CRAG drop/keep
tests/test_observability.py	LangSmith parent-span wiring (disabled in pytest)
tests/test_canonical_production.py	Runner dispatch, API response fields
tests/test_phase8_continuous_eval.py	Continuous eval loop, dashboard
tests/test_shadow_evaluation.py	Shadow paired comparison
tests/test_documentation_consistency.py	Doc ↔ code alignment
tests/test_production_safety.py	Auth, capacity, rate limits

The repository maintains an extensive automated test suite — do not rely on hardcoded counts in documentation.

16. Worked traces

```
# Canonical (preferred)
python -m src.cli ask "What is Self-RAG?" --mode canonical -v

# Tool-selected sources (API + MCP + optional calculator)
python -m src.cli ask "Who owns retriever-prod and what did experiment 42 conclude about chunking?" --mode source_tools -v

# Comparison / strategy selection visible in steps
python -m src.cli ask "Compare naive RAG and advanced RAG" --mode canonical -v

# HTTP
curl -s -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{"question": "What is Self-RAG?", "mode": "canonical"}' | jq .

# Streaming + pipeline debug events
curl -N http://localhost:8000/query/stream \
  -H "Content-Type: application/json" \
  -d '{"question": "What is Self-RAG?", "mode": "canonical"}'

# Phase 8 ops (requires API key when auth enabled)
curl -H "X-API-Key: $API_KEY" http://localhost:8000/ops/quality/dashboard
```

Agentic RAG — System Reference Manual (Deep Dive) · September 2026 · canonical-v1

Authoritative docs: [docs/DOCUMENTATION_MAP.md](#), [docs/ARCHITECTURE.md](#), [docs/LANGGRAPH.md](#), [docs/API.md](#), [docs/EVALUATION.md](#).

If code and this manual diverge, trust the code — then update this file.